

# Day 5 lab: audio and vision on the microcontroller

Naeemullah Khan

[naeemullah.khan@kaust.edu.sa](mailto:naeemullah.khan@kaust.edu.sa)



جامعة الملك عبد الله  
للعلوم والتقنية  
King Abdullah University of  
Science and Technology

KAUST Academy  
King Abdullah University of Science and Technology

October 7, 2026

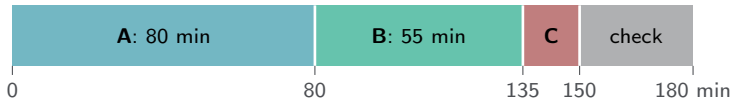
- ▶ **Goal:** your group has one audio model and one vision model on the XIAOML Kit: a keyword spotter that makes one event for each spoken keyword, and an image classifier that shows the class on the display.
- ▶ **Board:** the XIAOML Kit, with the microphone, the camera, and the display.
- ▶ **Deliverable:** two live demonstrations, and the file `report.md` with the measurement table and the Decision Log.

## From the lecture to the lab

The lecture gave the features, the two models, and the post-processing. In the lab, Edge Impulse Studio makes the features and the models. You write the post-processing, you tune it, and you measure the two models on the board.

**Key point:** The model comes from a tool in some minutes. The work of the engineer is the data before the model and the decisions after it.

# Plan of the three hours



| Part                    | Work                                                                                                               | Time   |
|-------------------------|--------------------------------------------------------------------------------------------------------------------|--------|
| A                       | Keyword spotting: data, training in Edge Impulse Studio, the int8 model on the kit, smoothing and threshold        | 80 min |
| B                       | Image classification: transfer learning on a prepared dataset, the int8 model on the kit, the class on the display | 55 min |
| C                       | Measure: latency and RAM of the two models                                                                         | 15 min |
| Check by the instructor |                                                                                                                    | 30 min |

Write each result in `report.md` when you get it.

## Hardware for each group

- ▶ XIAOML Kit: XIAO ESP32S3 Sense with the expansion board.
- ▶ USB-C data cable.
- ▶ Laptop with a microphone, for your own recordings.

## Software

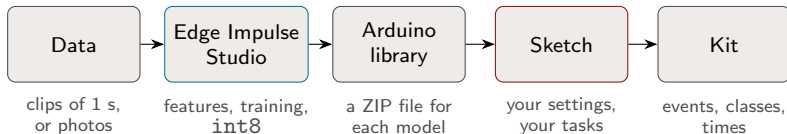
- ▶ Arduino IDE 2 with the esp32 core 3.3.12, and the library U8g2.
- ▶ One Edge Impulse account for each student.
- ▶ Python 3.10 or later for one script. It needs no package.

Lab files    Labs/day05/. Run the commands from this folder.

Build        Tools > PSRAM > OPI PSRAM for the two sketches

Sketches    sketches/kws\_stream/ and sketches/image\_classifier/

# The same path for the two models



- ▶ The Studio makes one library for each project. The library holds the feature code, the model, and the runtime.
- ▶ In each sketch, change the first `#include` line to the header of your library.
- ▶ In Deployment, select `Arduino library` and the model type `Quantized (int8)`.
- ▶ The first build of a library needs some minutes. Start it, and read the next step during the build.

**Key point:** On Days 3 and 4 you wrote the inference code. Today a tool writes it, and you write the code around it.

# Part A: keyword spotting (80 min)

| Step | Work                                                                               | Time   |
|------|------------------------------------------------------------------------------------|--------|
| 1    | Data: get the public clips with the script, upload them, and record your own words | 20 min |
| 2    | Impulse and features: a window of 1 s, the MFCC block                              | 10 min |
| 3    | Train the model, and test it in the Studio                                         | 10 min |
| 4    | Deploy: build the library, upload the sketch, count the events for 10 words        | 15 min |
| 5    | Tasks A1 and A2: the post-processing                                               | 10 min |
| 6    | Tune: three tests, and at least three rounds                                       | 15 min |

- ▶ The four classes: yes, no, unknown, and noise.
- ▶ The command `python3 host/get_keywords.py` writes 300 clips of each class into the folder `keywords/`.
- ▶ The Studio reads the label from the name of each file.

| Page             | Setting                                                                                                                           |
|------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Data acquisition | Upload data, with the automatic split between training and testing. Then record 10 s of yes and 10 s of no, and use Split sample. |
| Create impulse   | Window size 1000 ms, window increase 500 ms. Blocks: Audio (MFCC) and Classification.                                             |
| MFCC             | The default parameters. Generate the features.                                                                                    |
| Classifier       | The default model: two convolution layers with 8 and 16 filters. 100 training cycles, learning rate 0.005.                        |
| Model testing    | Classify all test clips.                                                                                                          |

- ▶ **You record:** the number of frames and coefficients, the accuracy, the class with the lowest accuracy, and the estimates of the Studio for the time, the RAM, and the flash.
- ▶ Compare the numbers of frames and coefficients with the example of the lecture: 49 frames of 13 coefficients.

Upload `sketches/kws_stream/kws_stream.ino`. The Serial Monitor at 115200 baud shows the settings, and then one line for each window:

```
Stride:      250 ms
Mean of:     3 windows
Threshold:   0.60
Suppression: 1000 ms
Task A1: not complete, no mean. Task A2: not complete, no suppression.
stride_ms,dsp_ms,classification_ms,no,noise,unknown,yes,event
...,...,...,...,...,...,...,none
...,...,...,...,...,...,...,yes (event 1)
```

- ▶ A task reads the microphone all the time. Each 250 ms, the loop runs the features and the model on the newest second.
- ▶ `stride_ms` is the real time between two windows. If it is larger than 250, the work for one window is longer than the stride.
- ▶ Before the two tasks, the sketch uses each window alone.

**Prediction.** You say “yes” 10 times, with pauses. How many events does the sketch give before the two tasks? Write your number in the report. Then count.

# Part A: tasks A1 and A2, the post-processing

The two tasks are in the tab `postprocess.h` of the sketch.

| Task | Function              | You write                                                                                                                                                   |
|------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A1   | <code>pp_mean</code>  | The mean of the last <code>n</code> results for one class. Use <code>pp_value(pp, class_index, back)</code> : <code>back</code> is 0 for the newest result. |
| A2   | <code>pp_event</code> | Rule 1: no event during the suppression time. Use <code>pp.had_event</code> , <code>pp.last_event_ms</code> , and <code>now_ms</code> .                     |

- ▶ Then set the flags `kTaskA1Complete` and `kTaskA2Complete` to `true`. The sketch prints the flags at the start.
- ▶ Rules 2 and 3 are in the file: the keyword class with the largest mean, and the threshold.
- ▶ Say “yes” 10 times again. Count the events. The target is 10.

**Key point:** With the example of the lecture, the code before the tasks gives 4 events, and the complete code gives 1 event.

| Test          | What you do                                       | What you count        |
|---------------|---------------------------------------------------|-----------------------|
| False accepts | One student reads a text with no keyword for 30 s | The events            |
| False rejects | Say yes 20 times, with pauses                     | The missed words      |
| Delay         | Say yes and look at the display                   | The time to the event |

- ▶ The four settings at the start of the sketch: `STRIDE_MS`, `SMOOTH_WINDOWS`, `THRESHOLD`, and `SUPPRESSION_MS`.
- ▶ Change one setting, upload, and repeat the three tests. Do at least three rounds. Write each round in the tuning table.
- ▶ **The check:** no event in 30 s of normal speech, and one event for each of 5 spoken words.

**Key point:** One setting for each round. With two changes in one round, you do not know which change gave the result.

# Part B: image classification (55 min)



| Step | Work                                                                            | Time   |
|------|---------------------------------------------------------------------------------|--------|
| 1    | Data: a second project, upload the three folders of <code>image_dataset/</code> | 10 min |
| 2    | Impulse: images of $96 \times 96$ , the transfer learning block                 | 5 min  |
| 3    | Train the model, and test it in the Studio                                      | 15 min |
| 4    | Deploy: build the library, upload the sketch                                    | 15 min |
| 5    | Test with the nine photos of <code>test_images/</code>                          | 10 min |

Source: Three of the 81 photos of the dataset, with the classes background, periquito, and robot. Photos: repository EdgeML-with-Raspberry-Pi by Marcelo Rovai.

| Page              | Setting                                                                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Data acquisition  | Upload each folder with its label, with the automatic split. It is not an object detection project.                                            |
| Create impulse    | Image size $96 \times 96$ . Blocks: Image and Transfer Learning (Images).                                                                      |
| Image             | Colour depth RGB. Generate the features.                                                                                                       |
| Transfer learning | Model MobileNetV2 $96 \times 96$ 0.35, 16 neurons in the last layer, dropout 0.1. 20 training cycles, learning rate 0.0005, data augmentation. |
| Model testing     | Classify all test images.                                                                                                                      |

- ▶ **You record:** the accuracy, and the estimates of the Studio for the inferencing time, the peak RAM, and the flash.
- ▶ Compare the peak RAM of the Studio with the peak of the lecture for this model: 135 KB.

Upload `sketches/image_classifier/image_classifier.ino`, with the header of your image library. The Serial Monitor shows for each frame:

```
Predictions (DSP: ... ms., Classification: ... ms., Anomaly: ... ms.):  
Timing (Capture: ... ms., Loop: ... ms.), free internal heap: ... bytes  
Predictions:  
  background: ...  
  periquito: ...  
  robot: ...  
Best: ... (... %), display: ...
```

- ▶ Open the nine photos of `test_images/` on the laptop screen, one after the other. Point the camera at the screen, so that the photo fills the image.
- ▶ The display shows a class only when three frames in sequence agree. Until then it shows ....
- ▶ Set `VOTE_FRAMES` to 1, upload, and compare.
- ▶ **You record:** the class of the display for each photo, with the two settings.

Source: The output shows the form of the lines.

| Number         | Keyword sketch                               | Image sketch                                             |
|----------------|----------------------------------------------|----------------------------------------------------------|
| Capture time   | Not necessary                                | Capture                                                  |
| Feature time   | <code>dsp_ms</code>                          | DSP                                                      |
| Inference time | <code>classification_ms</code>               | Classification                                           |
| Complete loop  | <code>stride_ms</code>                       | Loop                                                     |
| Flash          | Sketch uses ...<br>bytes of the build        | The same line                                            |
| RAM            | Free heap, and the peak<br>RAM of the Studio | free internal heap,<br>and the peak RAM of<br>the Studio |

Write the median of 10 lines for each time. Compare each measured time with the estimate of the Studio.

**Prediction.** For which of the two models is the feature time larger than the inference time? Write your answer before you read the Serial Monitor.

The instructor checks this at the end of the lab:

- ▶ ✓ The Serial Monitor of the keyword sketch shows Task A1: complete.  
Task A2: complete.
- ▶ ✓ The keyword sketch gives no event in 30 seconds of normal speech with no keyword.
- ▶ ✓ The keyword sketch gives one event for each spoken yes, in a test with 5 words.
- ▶ ✓ The image sketch shows the correct class for three test objects or test photos, one of each class.
- ▶ ✓ The report has the tuning table with at least three rounds.
- ▶ ✓ The report has the measurement table with the latency and the RAM of the two models.
- ▶ ✓ The Decision Log gives numbers and names one trade-off.

**Hand in:** the file `report.md`. Show the two demonstrations to the instructor.

Write about 100 words. State one design decision, give your measured numbers, and name the trade-off.

The two models go into one product: the voice-controlled camera of the lecture. The keyword model is stage 1, and the image model is stage 2.

- ▶ Which settings do you select for the post-processing of stage 1?
- ▶ Can one XIAO ESP32S3 run the two stages?

A good Decision Log:

- ▶ gives the false accepts, the missed words, and the delay of your settings from the tuning table,
- ▶ gives the times and the RAM of the two models from the measurement table,
- ▶ names what you lose with your decision. Example: fewer false accepts against a longer delay.

# If a part does not work, and common problems

| Problem                                                              | Fix or fallback                                                                                                          |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| The download of the keyword dataset fails                            | The instructor gives the file <code>keywords2.zip</code> . Put it into <code>downloads/</code> and run the script again. |
| The Studio is not available, or the training is not complete in time | Use the library ZIP file that the instructor gives. Write this in the report.                                            |
| The build of the library fails with the core 3.3.12                  | Install the core 2.0.17 and build again. The two sketches work with the two cores.                                       |
| The build prints<br><code>..._inferencing.h: No such file</code>     | Change the first <code>#include</code> line to the header of your library.                                               |
| not enough memory, or<br><code>Camera init failed</code>             | Set Tools > PSRAM > OPI PSRAM.                                                                                           |
| One word gives two or three events                                   | Complete task A2, or increase <code>SUPPRESSION_MS</code> .                                                              |
| A spoken word gives no event                                         | Use fewer windows, a lower threshold, or a smaller stride.                                                               |

The README of the lab has the complete list.

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. [mlsysbook.ai](https://mlsysbook.ai), CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. [github.com/Mjrovai/EdgeML\\_Made\\_Ease\\_ebook](https://github.com/Mjrovai/EdgeML_Made_Ease_ebook) and [github.com/Mjrovai/EdgeML-with-Raspberry-Pi](https://github.com/Mjrovai/EdgeML-with-Raspberry-Pi), GPL-3.0.
- ▶ *TinyML Made Easy: XIAO ESP32S3* by Marcelo Rovai. [github.com/Mjrovai/XIAO-ESP32S3-Sense](https://github.com/Mjrovai/XIAO-ESP32S3-Sense), Apache-2.0.